# UICopilot: Automating UI Synthesis via Hierarchical Code Generation from Webpage Designs

Yi Gui*†
Huazhong University of Science and Technology
Wuhan, China

Yao Wan‡†
Huazhong University of Science and Technology
Wuhan, China

Zhen Li*†
Huazhong University of Science and Technology
Wuhan, China

Zhongyi Zhang*
Huazhong University of Science and Technology
Wuhan, China

Dongping Chen
Huazhong University of Science and Technology
Wuhan, China

Hongyu Zhang
Chongqing University
Chongqing, China

Yi Su
Hubei University of Automotive Technology
Shiyan, China

Bohua Chen
Huazhong University of Science and Technology
Wuhan, China

Xing Zhou
Rabbitpre AI
Shenzhen, China

Wenbin Jiang†
Huazhong University of Science and Technology
Wuhan, China

Xiangliang Zhang
University of Notre Dame
Notre Dame, United States

## Abstract

Automating the synthesis of *User Interfaces* (UIs) plays a crucial role in enhancing productivity and accelerating the development lifecycle, reducing both development time and manual effort. Recently, the rapid development of *Multimodal Large Language Models* (MLLMs) has made it possible to generate front-end *Hypertext Markup Language* (HTML) code directly from webpage designs. However, real-world webpages encompass not only a diverse array of HTML tags but also complex stylesheets, resulting in significantly lengthy code. The lengthy code poses challenges for the performance and efficiency of MLLMs, especially in capturing the structural information of UI designs. To address these challenges, this paper proposes UICopilot, a novel approach to automating UI synthesis via hierarchical code generation from webpage designs. The core idea of UICopilot is to decompose the generation process into two stages: first, generating the coarse-grained HTML hierarchical structure, followed by the generation of fine-grained code. To validate the effectiveness of UICopilot, we conduct experiments on a real-world dataset, i.e., WebCode2M. Experimental results demonstrate that UICopilot significantly outperforms existing baselines in both automatic evaluation metrics and human evaluations. Specifically, statistical analysis reveals that the majority of human annotators prefer the webpages generated by UICopilot over those produced by GPT-4V.[1]

## CCS Concepts

• **Software and its engineering → Source code generation**.

## Keywords

UI Automation, Code Generation, Design to Code

**ACM Reference Format:**
Yi Gui, Yao Wan, Zhen Li, Zhongyi Zhang, Dongping Chen, Hongyu Zhang, Yi Su, Bohua Chen, Xing Zhou, Wenbin Jiang, and Xiangliang Zhang. 2025. UICopilot: Automating UI Synthesis via Hierarchical Code Generation from Webpage Designs. In *Proceedings of the ACM Web Conference 2025 (WWW '25), April 28-May 2, 2025, Sydney, NSW, Australia.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3696410.3714891

## 1 Introduction

Directly translating webpage designs into code significantly streamlines front-end development, reducing both development time and manual effort. This automation not only enhances developer productivity but also minimizes the risk of human error in code generation. To this end, several approaches have been explored. pix2code [2] was the first to generate *Domain-Specific Language* (DSL) code from webpage designs by training a neural network, validating its effectiveness on a synthetic dataset. The generated DSL code could then

*These authors contribute equally to this research.
†Also with National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Lab, Cluster and Grid Computing Lab, School of Computer Science and Technology,.
‡Yao Wan is the corresponding author (wanyao@hust.edu.cn).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
*WWW '25, Sydney, NSW, Australia*
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1274-6/25/04
https://doi.org/10.1145/3696410.3714891

[1]All the materials, including the source code and dataset, are available at: https://github.com/CGCL-codes/naturalcc/tree/main/examples/uicopilot.

be compiled into multiple front-end languages, including *Hypertext Markup Language* (HTML). Similarly, Sketch2code [31] leveraged deep learning and computer vision techniques to generate code from hand-drawn webpage sketches, further demonstrating the feasibility of automating design-to-code translation.

Recently, with the rapid advancement of *Multimodal Large Language Models* (MLLMs), generating high-quality UI code from webpage designs (e.g., screenshots) has become increasingly feasible. Building on this progress, several exploratory studies have been conducted, focusing on dataset creation and benchmarking [12, 14, 19, 36]. For example, WebSight [19] introduces a training dataset generated by an LLM, while Design2Code [36] offers a curated test dataset of 485 samples along with an automatic metric to assess the similarity between generated webpages and their original designs. WebCode2M [12] provides a large-scale real-world dataset for both training and evaluation of webpage generation. Similarly, Web2Code [47] presents a large-scale synthesized dataset and an MLLM-based evaluation framework. When datasets are available, these works also fine-tune MLLMs to better fit the task of design-to-code translation.

**Challenges and Motivation.** Despite the promising performance of the one-step generation approach, we are still far from fully automated UI code synthesis for real-world webpages. As noted in [36], the complexity of code generation increases significantly with the total number of HTML tags, the diversity of unique tags, and the depth of the *Document Object Model* (DOM) tree. When handling real-world webpage designs with intricate structures and a high number of unique HTML tags, existing MLLMs often suffer from notable declines in both performance and efficiency [36]. Specifically, the one-step generation approach faces two primary challenges.

**C1: The substantial length of the code that needs to be generated.** From our investigation, existing code generation tasks mainly focus on generating short code snippets or functions with fewer than a few hundred tokens. In contrast, real-world webpages consist not only of HTML but also complex *Cascading Style Sheets* (CSS), significantly increasing the overall code length. For instance, webpages in the Common Crawl dataset often contain tens of thousands of tokens, and even after extensive cleansing, they still average over 5,000 tokens [12]. This far exceeds the context window of most existing LLMs, posing substantial challenges for both training and inference. As a result, webpage generation resembles project-level development rather than the straightforward generation of isolated code snippets.

**C2: The complexity of generating deeply nested structures.** Furthermore, we observe that webpages typically consist of multiple layers of nested elements, making it particularly challenging to generate these intricate structures from high-resolution design diagrams. Prior studies [12, 25] have demonstrated that even GPT-4V struggles to accurately capture structural information, as evidenced by its performance on the pix2code test dataset—one of the simplest benchmarks for code generation from webpage designs.

**Our Work.** To tackle these challenges, this paper proposes UICopilot, a novel approach for automating UI synthesis through hierarchical code generation from webpage designs (screenshots). To mitigate the complexity of generating lengthy code, we decouple

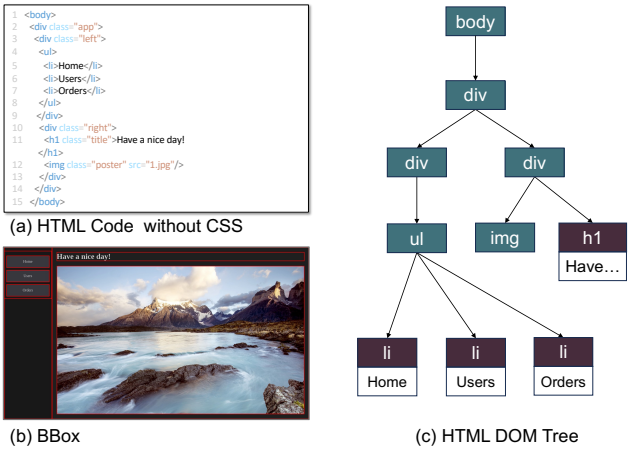



**Figure 1: The structural information of a webpage.**

the process into two stages: first, generating the coarse-grained hierarchical structure of HTML, and then producing the fine-grained code. Specifically, we introduce and train a *Vision Transformer* (ViT)-based structure model [6] to predict a coarse DOM tree that captures only node types, hierarchy, and bounding boxes (BBoxes). To streamline training, we employ a BBox-based pruning strategy that simplifies the dataset and reduces prediction complexity by removing HTML tags with a BBox area below a fixed threshold, while retaining only node type and hierarchy information. Using the predicted BBoxes of the coarse DOM tree's leaf nodes, the original design image is segmented into subregion images. These subregions are then sequentially processed by a code generation agent, which generates the corresponding HTML/CSS code and embeds it back into the DOM tree. Finally, the DOM tree and design image are fed into the code generation agent again to refine the global code while generating styles and attributes for the non-leaf nodes.

To assess the effectiveness of UICopilot, we conduct experiments on the real-world WebCode2M dataset, comparing it against several state-of-the-art baselines using three automatic metrics: CLIP [28], SSIM [44], and visual score [36]. The experimental results show that UICopilot significantly outperforms other baselines. Notably, the visual score of GPT-4V improves by 23%, 27%, and 48% when integrated into UICopilot across three test datasets of increasing complexity, demonstrating that UICopilot is particularly effective in handling complex webpage generation. Additionally, we present the webpages generated by our method and GPT-4V in randomized order to human annotators for evaluation. The results reveal that in over 60% of the cases, annotators prefer the webpages generated by our method, providing strong evidence for the effectiveness of UICopilot.

The primary contributions of this paper are as follows:

- We propose a novel approach UICopilot that decouples the generation of hierarchical structure and fine-grained code. To the best of our knowledge, we are the first to break the limitations of one-step webpage code generation by addressing the challenges of lengthy code generation and complex nested structures in real-world webpage synthesis.

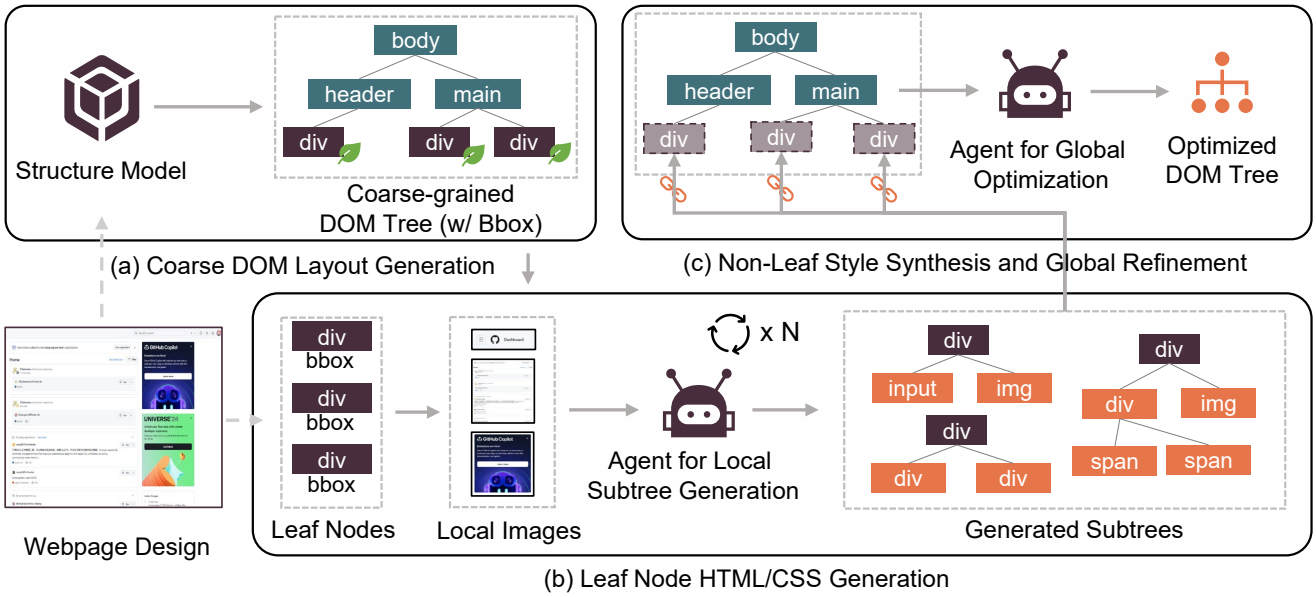



**Figure 2: An overview of our proposed UICopilot, which is composed of three components: (a) coarse DOM layout generation, (b) leaf node HTML/CSS generation, and (c) non-leaf style synthesis and global refinement.**

- We perform extensive experiments to evaluate the performance of UICopilot on the real-world WebCode2M dataset, and compare it with several state-of-the-art baselines. The results of automatic metrics and human evaluation show that UICopilot consistently outperforms other baselines.

## 2 Preliminaries

### 2.1 Hierarchy Structure of Webpage Designs

Typically, webpage code consists of three main components: HTML, CSS, and JavaScript. In this work, we focus on generating static webpages and therefore exclude JavaScript. The structural information in a webpage design diagram primarily comprises the hierarchical relationships between elements, along with their size and position. This information is crucial for ensuring high-quality webpage code generation and accurate final rendering. First, webpage code follows a hierarchical, nested structure, represented by the HTML DOM tree (Figure 1(c)), which serves as the backbone of the webpage. Second, the size and position of webpage elements are captured by **bounding boxes** (**BBox**) (Figure 1(b)), which define the primary layout structure. Our goal is to emphasize the preservation of a webpage's hierarchical structure to generate code that closely aligns with the original design.

### 2.2 Image to Code by MLLMs

Recently, MLLMs have made significant progress in understanding and generating multimodal content, including text [9], images [27], and audio [7]. In our scenario, we introduce two MLLMs: Pix2Struct-1.3B[20], which serves as the structural model for hierarchical structure prediction, and GPT-4V[26], which acts as the code generation agent. We select Pix2Struct as our base MLLM due to its ViT-based image encoder and its aspect-ratio-preserving scaling strategy, making it more robust to extreme aspect ratios and adaptable to varying sequence lengths and resolutions. Given an image, Pix2Struct first applies aspect-ratio-preserving scaling before splitting the image into fixed-size patches. These patches are then embedded and processed by the ViT encoder, which employs a self-attention mechanism to capture relationships between patches and understand the overall spatial arrangement of UI components. Finally, the model's Transformer-based text decoder takes the encoded representation and generates structured outputs, such as UI component expressions, functions, and locations.

## 3 UICopilot

### 3.1 Task Description

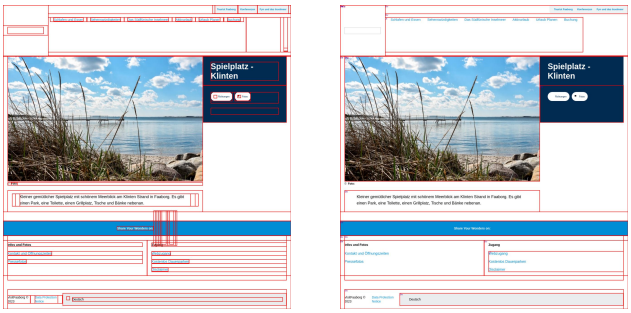
Given a high-resolution webpage design, such as a screenshot of an existing webpage, UICopilot aims to automatically generate the corresponding HTML and CSS code. The resulting webpage, once rendered, should closely resemble the input design in terms of layout, styling, and content. A crucial aspect of this process is accurately defining the structural elements of the page, including the type, dimensions, and positioning of components, as these factors directly influence the visual integrity of the generated webpage. Consequently, our work prioritizes the precise synthesis of structured and well-formed webpage layouts to ensure fidelity to the original design.

### 3.2 Overview

As shown in Figure 2, we decouple the webpage generation process into two stages: coarse DOM layout generation (Figure 2(a)) and fine-grained code synthesis (Figure 2(b&c)). Our approach is based on the assumption that "*when the elements and structure in a design*

```
{
    "type": "body",
    "bbox": [0, 0, 1280, 729],
    "children": [{
        "type": "div",
        "bbox": [39, 33, 1200, 658],
        "children": [{
            "type": "div",
            "bbox": [39, 158, 1200, 139]
        }]
    }]
}
```

**Figure 3: An example of DOM tree in JSON format.**




(a) Original bboxes (b) Pruned bboxes

**Figure 4: BBox-based data pruning.**

*image are sufficiently simple, MLLMs can effectively generate webpage code that captures both structure and detail*". By separating the generation of the DOM tree and positional information from the detailed local code, our approach alleviates the burden on LLMs when handling lengthy code generation. Additionally, prioritizing the DOM tree and positional details enhances the model's ability to accurately preserve the original webpage's structural elements, effectively addressing the challenges of generating lengthy code and capturing structural information, as mentioned before.

### 3.3 Coarse DOM Layout Generation

To generate webpage structure information more accurately and efficiently, we introduce and train a structure model specifically designed to predict a coarse-grained version of the HTML DOM tree, capturing only the node types, hierarchy, and BBoxes of the nodes. Additionally, we carefully design the data pruning and training procedures to optimize the performance.

**Structure Model.** In our scenario, high-resolution webpage designs appear in various resolutions, and directly cropping or resizing them often results in the loss of critical details, negatively impacting UI code generation. To address this, we draw inspiration from ViT's approach of segmenting images into multiple patches to retain as much information as possible. Based on this principle, we adopt Pix2Struct, a ViT-based model, as our core structure model. Pix2Struct is particularly well-suited for this task due to its robustness to extreme aspect ratios and its ability to dynamically adapt to varying sequence lengths and resolutions.

We formulate the structure prediction task as a sequence generation problem, following the next-token prediction paradigm. Given an input image $I$, which is a webpage design, the model produces a structured representation of the webpage's DOM tree in JSON format (exemplified in Figure 3). This JSON text encodes the DOM hierarchy as nested HTML elements, along with corresponding BBox attributes. The process begins by segmenting the input image $I$ into fixed-size patches, represented as $\{P_1, P_2, \ldots, P_N\}$. These patches are then processed by the ViT encoder, $\text{Enc}_\theta$, which transforms them into hidden state vectors as follows:

$$\{h_1, h_2, \ldots, h_N\} = \text{Enc}_\theta(\{P_1, P_2, \ldots, P_N\}) . \quad (1)$$

These hidden state vectors are subsequently fed into the decoder, $\text{Dec}_\phi$, a conventional Transformer-based text decoder, to predict the JSON representation of the DOM tree and their corresponding BBoxes. Leveraging the encoder's hidden states, the decoder generates the JSON text one token at a time. At each time step $t$, the decoder generates a probability distribution for the next token $y_t$, conditioned on all previous tokens $y_1, y_2, \ldots, y_{t-1}$. This prediction is computed using a softmax function applied to the decoder's hidden state $h_t$, which integrates information from both the encoder's hidden states $\{h_1, h_2, \ldots, h_N\}$ and the previously generated tokens, projected by a learned weight matrix $W$ as follows:

$$P(y_t \mid y_1, y_2, \ldots, y_{t-1}) = \text{softmax}(W \cdot h_t) . \quad (2)$$

**BBox-based Data Pruning.** Throughout our investigation, we find that the BBoxes of HTML elements in training datasets often contain a significant amount of noise, such as empty or very small elements, as well as elements representing visually hidden parts of the webpages or parts that are not displayed correctly (as illustrated in Figure 4(a)). The noise not only severely reduces the model's learning efficiency but also contributes little to the overall structural information. Therefore, we apply several heuristic rules to prune the original BBoxes along with their corresponding webpage elements: **(1)** We first remove BBoxes smaller than 3% of the total area, along with all their child nodes. We reasonably assume that for simple content within small areas, the MLLM can efficiently generate the corresponding high-quality structure and style code. **(2)** We eliminate BBoxes that contain only solid-colored pixels, along with all their child nodes. These BBoxes are usually empty and contain no actual web elements. **(3)** We discard webpage samples with fewer than 10 total BBoxes on the entire page, since an insufficient number of BBoxes typically indicates potential errors during processing.

Figure 4(b) presents an example of the pruned BBoxes, where most of the noise has been eliminated, revealing a clearer and simplified hierarchical structure. During model training, we find that this pruning method significantly accelerates model convergence while preserving as much of the original hierarchical structure of the webpage as possible. During the inference process of the structure model, this streamlined BBox information greatly improves both the generation quality and efficiency. We also apply pruning rules to the results generated by the structure model, primarily based on minimum area (`min_area`) of BBoxes and maximum depth (`max_depth`) of DOM tree. This ensures that the predicted DOM tree and BBoxes are concise and appropriate, setting a better stage for fine-grained code generation.

You are a skilled web developer specializing in building webpages.
**CONTEXT**
I am working on a project that involves converting webpage design images into functional code. Your task is to generate the corresponding HTML code for specific segments of the webpage based on the provided module names and images.
**OBJECTIVE**
Generate the partial HTML code based on the input webpage image and the initial node type.
**RESPONSE**
Provide the HTML code necessary to implement the module's functionality, including inline CSS.
**INITIALIZE**
In the upcoming messages, I will send you the webpage image and module name. Upon receiving them, please follow the above instructions to generate the corresponding HTML code (the highest-level node in the generated HTML tree should match the given initial node type).

**Figure 5: The prompt for leaf node HTML/CSS generation.**

You are a skilled web developer specializing in building webpages.
**CONTEXT**
I am working on a project that converts webpage design images into functional code. Your task is to adjust and optimize the styles of the already generated webpage code, based on the webpage image, without altering the original DOM tree structure of the code.
**OBJECTIVE**
Adjust and optimize the styles according to the input webpage image and original HTML code.
Do not modify the node types or hierarchical structure of the DOM tree in the original code!!!
Retain the original DOM tree nodes exactly as they are.
Do not change the image src attributes in the code.
**RESPONSE**
Adjust and optimize the CSS styles without altering the DOM tree structure in the original HTML code.
**INITIALIZE**
In the upcoming messages, I will send you the webpage image and the existing webpage code. Upon receiving them, please follow the above instructions to adjust and optimize the CSS styles in the original HTML code.

**Figure 6: The prompt for non-leaf style synthesis and global refinement.**

## 3.4 Fine-Grained Code Synthesis

While the generated coarse DOM tree contains the node types and hierarchy, it omits two crucial aspects: **(1)** The structural and style code for the local regions of the leaf nodes. Since the structure model is designed to predict a coarse-grained DOM tree, and this prediction is further pruned based on *min_area* and *max_depth*, the leaf nodes in the DOM tree actually serve as parent nodes for subregions that still contain sub-trees requiring further prediction. **(2)** The attributes and styles of the non-leaf nodes within the coarse DOM tree. To address these two missing elements, we have devised the following two steps to complete the generation process.

**HTML/CSS Generation of Leaf Node.** In practice, we employ GPT-4V as our local code generation agent, leveraging its flexibility and robust code generation capabilities. Since the BBoxes contain the size and position information of the nodes, we use the BBoxes of the leaf nodes in the predicted coarse DOM tree to crop images of the corresponding subregions from the original image. These segmented images are then fed individually into the code generation agent to predict the corresponding HTML/CSS code for each subregion. After obtaining the HTML/CSS code for the leaf nodes, we embed them back into the corresponding leaf nodes of the coarse DOM tree. By doing so, we isolate the visual content associated with each leaf node, allowing the agent to generate accurate HTML/CSS code for these specific regions one by one, significantly alleviating the burden of generating lengthy code and improving the generation quality. We have carefully designed a prompt that instructs the agent to generate the corresponding HTML/CSS code based on the subregion image and the parent node type, as shown in Figure 5.

**Non-Leaf Style Synthesis and Global Refinement.** At this stage, we provide the code from the previous step along with the full design image to the agent, instructing it to add styles and other attributes to the non-leaf nodes of the coarse DOM tree. The agent analyzes the global layout and visual elements to infer styling details like fonts, colors, margins, paddings, and other CSS properties, ensuring that the final code reflects both the structural hierarchy and the visual aesthetics of the original design. Additionally, the agent refines the global code while preserving the input DOM tree. The prompt used in this process is shown in Figure 6.

**Table 1: A statistical comparison between both WebSight and WebCode2M. The statistical data of the two is referred to [36].**

| | WebSight | WebCode2M |
|---|---|---|
| Purpose | Training | Training&Tesing |
| Source | Synthetic | Real-World (Common Crawl) |
| Size | 0.8M | 3.1M |
| Avg. Len (tokens) | 647±216 | 4661±2006 |
| Avg. Tags | 19±8 | 188±80 |
| Avg. DOM Depth | 5±1 | 15±5 |
| Avg. Unique Tags | 10±3 | 24±6 |

# 4 Experiments and Analysis

## 4.1 Datasets

**Training Dataset.** We utilize two datasets, WebSight v0.1 and WebCode2M, for training the structure model, each demonstrating distinct characteristics across various indicators. As shown in Table 1, compared to the WebSight dataset, WebCode2M's data is more complex, possesses a richer variety of styles and tag types, and is significantly longer, making it closer to real-world HTML code. Furthermore, WebCode2M provides the page's BBox information directly, which is essential for the training of our structure model. For the WebSight dataset, we also extract the BBox information prior to training.

**Test Dataset.** We evaluate our framework on the WebCode2M test datasets. WebCode2M test datasets are composed of three subsets: WebCode2M-Short, WebCode2M-Mid, and WebCode2M-Long. These subsets are created by splitting the data according to the length range of the ground-truth HTML code. The length ranges of the ground-truth HTML code for these three subsets are [551, 2045], [2052,4085], and [4098,10990], respectively. Each subset contains 256 samples.

**Table 2: The performance breakdown on the visual metrics.**

| Model | WebCode2M-Short | | | WebCode2M-Mid | | | WebCode2M-Long | | |
|---|---|---|---|---|---|---|---|---|---|
| | Visual Score | CLIP | SSIM | Visual Score | CLIP | SSIM | Visual Score | CLIP | SSIM |
| WebSight VLM-7B | 0.57 (±0.24) | 0.69 (±0.12) | 0.62 (±0.17) | 0.52 (±0.23) | 0.67 (±0.11) | 0.59 (±0.16) | 0.48 (±0.27) | 0.64 (±0.11) | 0.61 (±0.15) |
| Design2Code | 0.75 (±0.14) | 0.68 (±0.10) | 0.58 (±0.15) | 0.69 (±0.23) | 0.70 (±0.10) | 0.56 (±0.14) | 0.61 (±0.28) | 0.68 (±0.10) | 0.61 (±0.11) |
| CogAgent-Chat | 0.46 (±0.31) | 0.68 (±0.11) | 0.59 (±0.15) | 0.40 (±0.31) | 0.66 (±0.10) | 0.58 (±0.14) | 0.39 (±0.30) | 0.65 (±0.10) | 0.60 (±0.13) |
| LLaVA-v1.5-7B | 0.43 (±0.27) | 0.63 (±0.11) | **0.65 (±0.17)** | 0.21 (±0.28) | 0.63 (±0.10) | **0.65 (±0.14)** | 0.19 (±0.27) | 0.61 (±0.10) | **0.66 (±0.12)** |
| GPT-4V | 0.68 (±0.32) | 0.74 (±0.10) | 0.61 (±0.14) | 0.65 (±0.33) | 0.71 (±0.10) | 0.55 (±0.12) | 0.62 (±0.35) | 0.67 (±0.10) | 0.57 (±0.11) |
| UICopilot | **0.84 (±0.18)** | **0.77 (±0.11)** | 0.60 (±0.13) | **0.83 (±0.17)** | **0.77 (±0.10)** | 0.57 (±0.12) | **0.78 (±0.24)** | **0.74 (±0.10)** | 0.60 (±0.11) |

## 4.2 Evaluation Metrics

**CLIP Similarity [28].** CLIP is a multi-modal model trained using a contrastive objective on a large dataset of millions of internet text-image pairs. It learns to align images with their textual descriptions in a shared representation space. The latent vectors generated by the CLIP model capture the semantic information of the inputs. As a result, the cosine similarity between these vectors, which calculates the cosine of the angle between them, effectively quantifies the degree of similarity between the images.

**SSIM (*Structural Similarity Index*) [44].** It considers changes in structural, luminance, and contrast information in the images.

**Visual Score [36].** It is used to assess the degree of alignment between low-level elements based on their appearance. The scores primarily evaluate the match ratio between reference and candidate blocks, as well as their similarity at the block level, considering factors such as color, text, and position.[2]

## 4.3 Baselines

The baselines in our work fall into two categories: MLLMs specialized for webpage generation and general-purpose MLLMs. The specialized MLLMs include:

- **WebSight VLM-8B [19].** Hugging Face's WebSight utilizes its training dataset and the DoRA [23] mechanism to fine-tune a VLM that has been pre-trained on image/text pairs.
- **Design2Code-18B [36].** Stanford's Design2Code is also fine-tuned on the WebSight dataset. However, it adopts CogAgent [16] as its base model and utilizes LoRA [17] as the fine-tuning method to accelerate the training process.

The general-purpose (prompt-based) MLLMs include:

- **CogAgent-Chat-18B [16].** CogAgent-Chat-18B is a general MLLM that supports both low- and high-resolution images. Notably, it performs well on webpage navigation, requiring only screenshots. We input the screenshot and a simple prompt "*write an HTML code*" to generate the webpage, similar to Design2Code.
- **GPT-4V [26].** GPT-4V has shown remarkable capabilities in image comprehension. It also possesses the unique ability to generate code from images. We refer to the prompt of the well-known open-source project *screenshot-to-code* [3] on GitHub, with slight modifications.
- **LLaVA-v1.5-7B [22].** LLaVA-v1.5-7B is an MLLM that connects a vision encoder and an LLM for general-purpose visual and language understanding. We use the same prompt as GPT-4V to generate webpages from images.

[2]We use the visual score from the first version of their paper, which may differ from the latest version, especially with regard to the sub-indicator *text color*.
[3]https://github.com/abi/screenshot-to-code

## 4.4 The Effectiveness of UICopilot

**Overall Performance.** Table 2 presents the performance breakdown of UICopilot and the baselines on the WebCode2M test datasets. From the table, we observe that our method's visual score and CLIP similarity significantly outperform all the baselines across the three test datasets. This leading performance in the CLIP metric indicates that the webpages generated by UICopilot are more visually similar to the original ones in terms of overall appearance and features. In terms of SSIM, LLaVA holds a slight advantage over the other models, suggesting that the webpages it generates may be more closely aligned with the originals concerning visual luminance, contrast, and structure. However, the differences among all models in this metric are not significant. Our UICopilot also performs well on this metric compared to the other models.

Notably, while UICopilot utilizes GPT-4V for generating fine-grained code, it still outperforms GPT-4V in terms of visual score by 23%, 27%, and 48% across the three test datasets, respectively. Additionally, UICopilot exhibits significantly lower variance in visual scores, indicating higher stability. Moreover, it is worth noting that as the length of the sample webpages in the test datasets increases, the advantage of UICopilot over GPT-4V grows substantially. This further demonstrates that UICopilot's architecture—which focuses on decoupling the structure parsing from the fine-grained code generation processes—reduces complexity and results in more accurate HTML/CSS code generation from design images.

**Visual Score Breakdown.** The visual score is a composite metric consisting of five sub-indicators: block-level color, text, position, text color, and CLIP similarity information. In Figure 7, we provide a detailed analysis of these sub-indicators. To further explore the contribution of the refinement process in our framework, we introduce UICopilot without the global refinement (denoted as UICopilot w/o opt) for comparison. As shown in Figure 7, UICopilot without the refinement already outperforms GPT-4V in terms of CLIP similarity, text, position, and text color. This demonstrates that the input coarse-grained HTML DOM tree and BBoxes effectively enhance GPT-4V's generation capability, particularly evident in the significant improvement in the text color indicator. However, we observe that UICopilot without refinement performs worse than GPT-4V in the block-level color indicator. Our investigation suggests that strictly adhering to the coarse-grained HTML DOM tree and BBoxes without refinement can lead to missing styles or

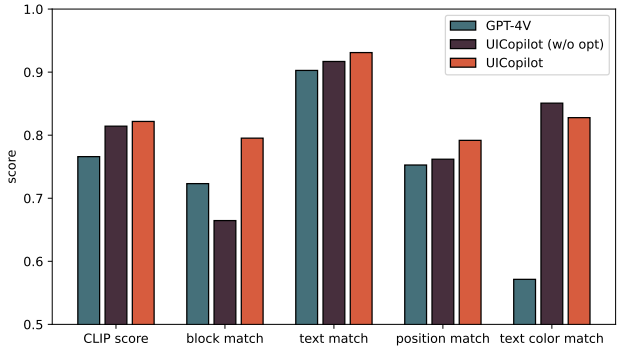



**Figure 7: Detailed sub-indicators of visual score.**

errors in the upper layers of nodes, negatively impacting the block-level color. After incorporating the refinement process, as seen in Figure 7, there is a significant improvement in the block-level color indicator, highlighting the effectiveness of the refinement process.

## 4.5 The Influence of `min_area` and `max_depth`

As described in Section 3.3, we prune the generated results by the structure model based on two parameters: the minimum area of BBox (`min_area`) and the maximum depth of the DOM tree (`max_depth`). To search for the optimal values of these parameters, we conduct a grid search to explore different combinations. However, since performing inference on the entire test dataset is time-consuming, we optimize the grid search process by constraining the search space based on empirical experience: [10%, 20%, 30%, Unlimited] for `min_area` and [4, 5, 6, Unlimited] for `max_depth`. We focus on the metric with the highest relative improvement in our method—the visual score—for further experiments. By analyzing its sub-metrics under different combinations of `min_area` and `max_depth`, we obtain the results shown in Figure 8. Our analysis reveals a consistent linear trend across nearly all sub-metrics: as both `max_depth` and `min_area` decrease, the visual score improves, indicating enhanced visual quality in the generated results. Specifically, we find that setting `max_depth` to 4 and `min_area` to 10% achieves a well-balanced local optimum for visual matching.

## 4.6 Human Evaluation

From Table 2, we observe minimal differences between UICopilot and GPT-4V in terms of CLIP similarity and SSIM, both of which primarily measure overall image similarity. To address the question, "*which model generates webpages that are closer to the original design?*", we conduct a human evaluation study. Specifically, we present six annotators with pairs of webpages generated by UICopilot and GPT-4V, shuffling the samples in each pair to eliminate bias. Annotators are asked to select the webpage they find most aligned with the original design. As shown in Figure 9, over 60% of selections favored UICopilot, providing strong evidence of its effectiveness in generating webpages that more closely resemble the original designs.

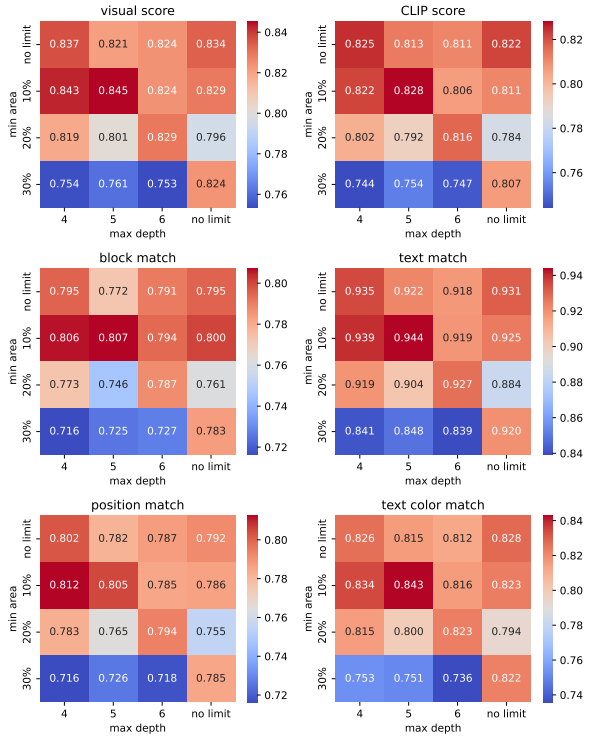



**Figure 8: Grid search results of parameter optimization.**

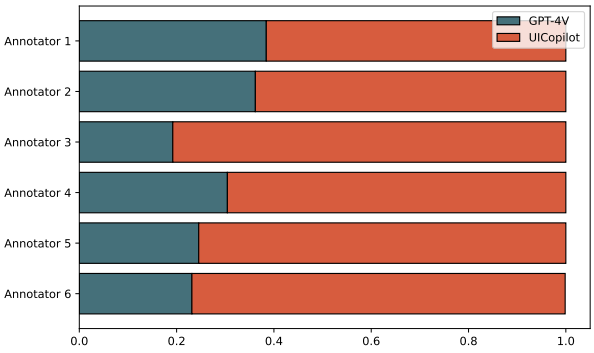
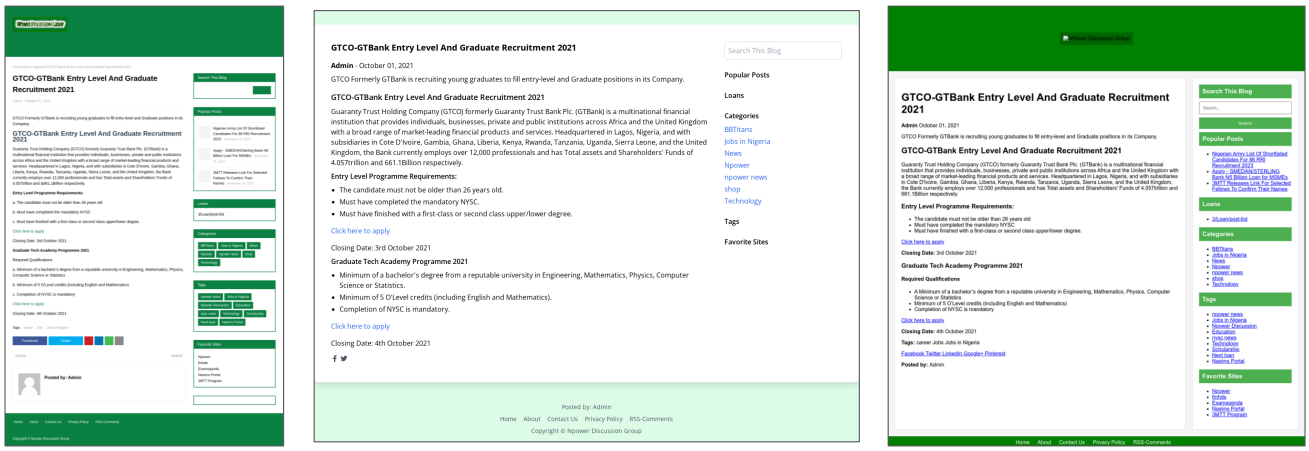



**Figure 9: The preference of human annotators.**

## 4.7 Case Study

To highlight the advantages of UICopilot in webpage generation, we present a representative example in Figure 10. The images, from left to right, show the original webpage, the one generated by GPT-4V, and the one generated by UICopilot. Both GPT-4V and UICopilot capture the footer, body, and header, with text content close to the original. However, GPT-4V simplifies the small blocks on the right side, while UICopilot accurately replicates these structural details. This demonstrates that decomposing webpage synthesis into coarse-grained DOM tree generation and fine-grained localized code enhances structural accuracy. Despite this, both models still




(a) Original (b) GPT-4V (c) UICopilot

**Figure 10: A representative example of the generated webpages by GPT-4V and UICopilot.**

miss finer details such as text, colors, and section sizes, suggesting that post-generation edits and localized fixes could improve webpage quality, making them valuable areas for future exploration.

## 5 Related Work

**Image Representation Learning.** Learning effective image representations is fundamental to a wide range of computer vision tasks, including image-to-text applications such as image captioning [38], image-to-code translation [36], image classification [34], object detection [10], and semantic segmentation [35]. Extensive research has explored various strategies for extracting meaningful latent representations from images. For instance, Chen et al. [5] leveraged contrastive learning to train image encoders on large-scale datasets, improving feature extraction through self-supervision. ViT [6] introduced an innovative paradigm by segmenting images into fixed-size patches and processing them using a Transformer-based architecture, enabling adaptability to varying resolutions. More recently, diffusion models [15, 32] have emerged as powerful tools for image representation and visual data generation.

**Code Generation.** Neural language models have seen remarkable progress in code intelligence [39], enabling advancements in key tasks such as code summarization [41, 42], code search [40], and code generation [3, 37]. With the rapid evolution of LLMs in text generation, numerous code-specific LLMs have emerged, including CodeT5+[43], InCoder[8], StarCoder [21], Code Llama [33], WizardCoder [24], Qwen-Coder [18], and DeepSeek-Coder [13]. These models have revolutionized software development by powering production tools such as Copilot [11], CodeWhisperer [1], and Replit [30], streamlining coding workflows and enhancing developer productivity. While much research has focused on general-purpose LLMs for code generation, this paper explores a distinct challenge: generating code directly from webpage designs. This specialized task requires a nuanced understanding of UI structures, layout constraints, and code synthesis, setting it apart from traditional code generation paradigms.

**Image to Code.** There are various works focused on generating code from images. For example, Sketch2code [31] generates website code from wireframe sketches using two approaches: a computer vision-based method to identify elements and structures, and a deep learning-based method to translate website sketches into normalized webpages for code generation. Pix2Struct [20], pretrained to predict simplified HTML from masked website screenshots, has significantly improved visual language understanding across nine tasks in four domains. Wu et al. [45] tackled the problem of screen parsing by predicting UI hierarchy graphs from screenshots. They utilized Faster-RCNN [29] for image encoding and an LSTM-based attention mechanism to construct the graph nodes and edges. Additionally, some studies focus on curating specialized training datasets [12, 19] to improve MLLMs' capabilities in UI generation, while others aim to establish benchmarks and evaluation metrics [4, 36, 46] for assessing these models.

## 6 Conclusion

In this paper, we have proposed UICopilot, a novel approach for code generation from webpage designs by decoupling the process into two stages: coarse DOM tree prediction and fine-grained code synthesis. Specifically, we employ a ViT-based structure model to predict the DOM tree and enhance training with a BBox-based data pruning strategy. Using the coarse DOM trees and BBoxes predicted by the structure model, we break down the code generation process into two steps: generating code snippets for segmented subregions of the image and assembling them. This approach avoids the complexity of generating long code in a single step. Experimental results and human annotator feedback validate the effectiveness of our proposed approach.

## Acknowledgments

This work is supported by the National Key Research and Development Program of China under Grant No. 2022YFB4501400, and the National Natural Science Foundation of China under Grant No. 62372199.

## References


[1] Amazon AWS. 2022. AI Code Generator - Amazon CodeWhisperer - AWS. https://aws.amazon.com/codewhisperer/.

[2] Tony Beltramelli. 2018. pix2code: Generating Code from a Graphical User Interface Screenshot. In *Proceedings of the ACM SIGCHI Symposium on Engineering Interactive Computing Systems, EICS 2018, Paris, France, June 19-22, 2018*. ACM, 3:1–3:6.

[3] Zhangqian Bi, Yao Wan, Zheng Wang, Hongyu Zhang, Batu Guan, Fangxin Lu, Zili Zhang, Yulei Sui, Hai Jin, and Xuanhua Shi. 2024. Iterative Refinement of Project-Level Code Context for Precise Code Generation with Compiler Feedback. In *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024*. Association for Computational Linguistics, 2336–2353.

[4] Dongping Chen, Yue Huang, Siyuan Wu, Jingyu Tang, Huichi Zhou, Qihui Zhang, Zhigang He, Yilin Bai, Chujie Gao, Liuyi Chen, et al. [n. d.]. GUI-WORLD: A GUI-oriented Video Dataset for Multimodal LLM-based Agents. In *Workshop on Video-Language Models@ NeurIPS 2024*.

[5] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey E. Hinton. 2020. A Simple Framework for Contrastive Learning of Visual Representations. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, 1597–1607.

[6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *Proceedings of 9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net.

[7] Zach Evans, CJ Carr, Josiah Taylor, Scott H. Hawley, and Jordi Pons. 2024. Fast Timing-Conditioned Latent Audio Diffusion. In *Proceedings of Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net.

[8] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Scott Yih, Luke Zettlemoyer, and Mike Lewis. 2023. InCoder: A Generative Model for Code Infilling and Synthesis. In *Proceedings of The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.

[9] Zhongtian Fu, Kefei Song, Luping Zhou, and Yang Yang. 2024. Noise-Aware Image Captioning with Progressively Exploring Mismatched Words. In *Proceedings of Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2014, February 20-27, 2024, Vancouver, Canada*, Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan (Eds.). AAAI Press, 12091–12099.

[10] Ross B. Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. 2014. Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation. In *Proceedings of 2014 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2014, Columbus, OH, USA, June 23-28, 2014*. IEEE Computer Society, 580–587.

[11] GitHub. 2022. GitHub Copilot - Your AI pair programmer. https://github.com/features/copilot/.

[12] Yi Gui, Zhen Li, Yao Wan, Yemin Shi, Hongyu Zhang, Yi Su, Bohua Chen, Dongping Chen, Siyuan Wu, Xing Zhou, et al. 2024. WebCode2M: A Real-World Dataset for Code Generation from Webpage Designs. In *Proceedings of International World Wide Web Conference, WWW 2024, Singapore, May 13-17, 2024*.

[13] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming–The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).

[14] Hongcheng Guo, Wei Zhang, Junhao Chen, Yaonan Gu, Jian Yang, Junjia Du, Binyuan Hui, Tianyu Liu, Jianxin Ma, Chang Zhou, and Zhoujun Li. 2024. IW-Bench: Evaluating Large Multimodal Models for Converting Image-to-Web. *CoRR* abs/2409.18980 (2024).

[15] Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising Diffusion Probabilistic Models. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*.

[16] Wenyi Hong, Weihan Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, and Jie Tang. 2024. CogAgent: A Visual Language Model for GUI Agents. In *Proceedings of IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024, Seattle, WA, USA, June 16-22, 2024*. IEEE, 14281–14290.

[17] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *Proceedings of The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net.

[18] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).

[19] Hugo Laurençon, Léo Tronchon, and Victor Sanh. 2024. Unlocking the conversion of Web Screenshots into HTML Code with the WebSight Dataset. *CoRR* abs/2403.09029 (2024). arXiv:2403.09029

[20] Kenton Lee, Mandar Joshi, Iulia Raluca Turc, Hexiang Hu, Fangyu Liu, Julian Martin Eisenschlos, Urvashi Khandelwal, Peter Shaw, Ming-Wei Chang, and Kristina Toutanova. 2023. Pix2Struct: Screenshot Parsing as Pretraining for Visual Language Understanding. In *Proceedings of International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 18893–18912.

[21] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy V, Jason T. Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Nour Fahmy, Urvashi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. 2023. StarCoder: may the source be with you! *Trans. Mach. Learn. Res.* 2023 (2023).

[22] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023.

[23] Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. 2024. DoRA: Weight-Decomposed Low-Rank Adaptation. In *Proceedings of Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net.

[24] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2024. WizardCoder: Empowering Code Large Language Models with Evol-Instruct. In *Proceedings of The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.

[25] Kevin Moran, Carlos Bernal-Cárdenas, Michael Curcio, Richard Bonett, and Denys Poshyvanyk. 2020. Machine Learning-Based Prototyping of Graphical User Interfaces for Mobile Apps. *IEEE Trans. Software Eng.* 46, 2 (2020), 196–221.

[26] OpenAI. 2023. GPT-4 Technical Report. *CoRR* abs/2303.08774 (2023). arXiv:2303.08774

[27] Dustin Podell, Zion English, Kyle Lacey, Andreas Blattmann, Tim Dockhorn, Jonas Müller, Joe Penna, and Robin Rombach. 2024. SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis. In *Proceedings of The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.

[28] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event (Proceedings of Machine Learning Research, Vol. 139)*. PMLR, 8748–8763.

[29] Shaoqing Ren, Kaiming He, Ross B. Girshick, and Jian Sun. 2015. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. (2015), 91–99.

[30] Replit. 2023. Meet Replit AI, your partner in code. https://replit.com/ai/.

[31] Alex Robinson. 2019. Sketch2code: Generating a website from a paper mockup. *CoRR* abs/1905.13750 (2019). arXiv:1905.13750

[32] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. High-Resolution Image Synthesis with Latent Diffusion Models. In *Proceedings of IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*. IEEE, 10674–10685.

[33] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).

[34] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael S. Bernstein, Alexander C. Berg, and Li Fei-Fei. 2015. ImageNet Large Scale Visual Recognition Challenge. *Int. J. Comput. Vis.* 115, 3 (2015), 211–252.

[35] Evan Shelhamer, Jonathan Long, and Trevor Darrell. 2017. Fully Convolutional Networks for Semantic Segmentation. *IEEE Trans. Pattern Anal. Mach. Intell.* 39, 4 (2017), 640–651.

[36] Chenglei Si, Yanzhe Zhang, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. 2024. Design2Code: How Far Are We From Automating Front-End Engineering? *CoRR* abs/2403.03163.

[37] Zhihong Sun, Yao Wan, Jia Li, Hongyu Zhang, Zhi Jin, Ge Li, and Chen Lyu. 2024. Sifting through the Chaff: On Utilizing Execution Feedback for Ranking the Generated Code Candidates. In *Proceedings of the 39th IEEE/ACM International*

*Conference on Automated Software Engineering, ASE 2024, Sacramento, CA, USA, October 27 - November 1, 2024*. ACM, 229–241.

[38] Oriol Vinyals, Alexander Toshev, Samy Bengio, and Dumitru Erhan. [n. d.]. Show and tell: A neural image caption generator. In *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2015, Boston, MA, USA, June 7-12, 2015, pages = 3156–3164, publisher = IEEE Computer Society, year = 2015.*

[39] Yao Wan, Zhangqian Bi, Yang He, Jianguo Zhang, Hongyu Zhang, Yulei Sui, Guandong Xu, Hai Jin, and Philip Yu. 2024. Deep Learning for Code Intelligence: Survey, Benchmark and Toolkit. *ACM Computing Survey* 56, 12, Article 309 (Oct. 2024), 41 pages. doi:10.1145/3664597

[40] Yao Wan, Jingdong Shu, Yulei Sui, Guandong Xu, Zhou Zhao, Jian Wu, and Philip Yu. 2019. Multi-modal attention network learning for semantic source code retrieval. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 13–25.

[41] Yao Wan, Zhou Zhao, Min Yang, Guandong Xu, Haochao Ying, Jian Wu, and Philip S Yu. 2018. Improving automatic source code summarization via deep reinforcement learning. In *Proceedings of the 33rd ACM/IEEE international conference on automated software engineering*. 397–407.

[42] Wenhua Wang, Yuqun Zhang, Yulei Sui, Yao Wan, Zhou Zhao, Jian Wu, S Yu Philip, and Guandong Xu. 2020. Reinforcement-learning-guided source code summarization using hierarchical attention. *IEEE Transactions on software Engineering* 48, 1 (2020), 102–119.

[43] Yue Wang, Hung Le, Akhilesh Gotmare, Nghi D. Q. Bui, Junnan Li, and Steven C. H. Hoi. 2023. CodeT5+: Open Code Large Language Models for Code Understanding and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*. Association for Computational Linguistics, 1069–1088.

[44] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. 2004. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing* 13, 4 (2004), 600–612.

[45] Jason Wu, Xiaoyi Zhang, Jeffrey Nichols, and Jeffrey P. Bigham. 2021. Screen Parsing: Towards Reverse Engineering of UI Models from Screenshots. In *Proceedings of UIST '21: The 34th Annual ACM Symposium on User Interface Software and Technology, Virtual Event, USA, October 10-14, 2021*. ACM, 470–483.

[46] Jingyu Xiao, Yuxuan Wan, Yintong Huo, Zhiyao Xu, and Michael R Lyu. 2024. Interaction2Code: How Far Are We From Automatic Interactive Webpage Generation? *arXiv preprint arXiv:2411.03292* (2024).

[47] Sukmin Yun, Haokun Lin, Rusiru Thushara, Mohammad Qazim Bhat, Yongxin Wang, Zutao Jiang, Mingkai Deng, Jinhong Wang, Tianhua Tao, Junbo Li, Haonan Li, Preslav Nakov, Timothy Baldwin, Zhengzhong Liu, Eric P. Xing, Xiaodan Liang, and Zhiqiang Shen. 2024. Web2Code: A Large-scale Webpage-to-Code Dataset and Evaluation Framework for Multimodal LLMs. *CoRR* abs/2406.20098 (2024). arXiv:2406.20098